



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Compiladores

2027-1

Profesora: L. en C.C. Yessica Janeth Pablo Martínez

Ayudante de Laboratorio: Pedro Yamil Salazar González

Práctica 4

Semestre 2027-1

1. Introducción

En las prácticas anteriores se implementaron el analizador léxico de MiniC y un analizador sintáctico descendente recursivo.

El parser desarrollado recibe la secuencia de tokens producida por el analizador léxico y verifica que estos se encuentren organizados de acuerdo con la gramática del lenguaje.

Hasta este momento, el analizador sintáctico responde únicamente la siguiente pregunta:

¿El programa está sintácticamente bien escrito?

Por ejemplo, para el programa:

```
1 int x;  
2 x = 5 + 2;
```

el parser puede determinar que las declaraciones, asignaciones y expresiones cumplen con las producciones de MiniC.

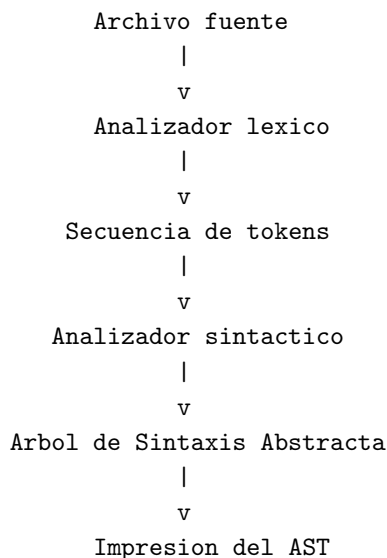
Sin embargo, después de validar la sintaxis, el compilador necesita conservar una representación estructurada del programa para utilizarla en las siguientes etapas.

En esta práctica se extenderá el parser para responder una nueva pregunta:

¿Cómo está construido el programa?

Para ello se implementará un **Árbol de Sintaxis Abstracta**, denominado comúnmente *AST* por sus siglas en inglés, *Abstract Syntax Tree*.

El nuevo flujo del compilador será:



A diferencia de la Práctica 3, las funciones del parser ya no deberán limitarse a consumir tokens. Ahora deberán construir y devolver nodos que representen las declaraciones, sentencias y expresiones reconocidas.

Por ejemplo, la función:

```
1 void parse_expression(Parser *parser);
```

deberá transformarse en una función equivalente a:

```
1 ASTNode *parse_expression(Parser *parser);
```

La estructura construida en esta práctica será utilizada posteriormente por el análisis semántico y la generación de código.

2. Objetivo general

Extender el analizador sintáctico descendente recursivo de MiniC para construir un Árbol de Sintaxis Abstracta que represente la estructura esencial de un programa sintácticamente válido.

3. Objetivos específicos

Al finalizar esta práctica, el estudiante será capaz de:

- distinguir entre un Árbol de Sintaxis Concreta y un Árbol de Sintaxis Abstracta;
- representar programas mediante estructuras jerárquicas;
- diseñar una jerarquía de nodos para declaraciones, sentencias y expresiones;
- modificar las funciones del parser para devolver nodos;
- construir árboles respetando la precedencia y asociatividad de los operadores;
- conservar información relevante de los tokens dentro del AST;
- representar listas de sentencias;
- imprimir el árbol de forma jerárquica;
- liberar correctamente la memoria ocupada por el AST;
- mantener separados los módulos del lexer, parser y AST;
- reutilizar la validación sintáctica implementada previamente;
- preparar una representación intermedia para el análisis semántico.

4. Antecedentes

4.1. Árbol de Sintaxis Concreta

Un Árbol de Sintaxis Concreta, o *CST*, representa de manera explícita todas las producciones gramaticales utilizadas para reconocer una entrada.

Considere la expresión:

```
1 x = a + b * c;
```

Una representación simplificada de su CST podría ser:

```
Assignment
|
|-- IDENTIFIER(x)
|-- ASSIGN
|-- Expression
|  |
|  |-- Term
|  |  '-- IDENTIFIER(a)
|  |
|  |-- PLUS
|  |
|  '-- Term
|     |
|     |-- Factor
|     |  '-- IDENTIFIER(b)
|     |
|     |-- STAR
|     |
|     '-- Factor
|        '-- IDENTIFIER(c)
|-- SEMICOLON
```

El CST conserva elementos necesarios para describir la derivación gramatical, incluyendo:

- símbolos terminales;
- símbolos no terminales;
- delimitadores;
- producciones intermedias;
- signos de puntuación.

Esta representación resulta útil para estudiar la gramática, pero contiene información que normalmente no es necesaria para las siguientes etapas del compilador.

4.2. Árbol de Sintaxis Abstracta

El Árbol de Sintaxis Abstracta conserva únicamente los elementos esenciales para representar el significado estructural del programa.

Para la misma entrada:

```
1 x = a + b * c;
```

el AST podría representarse como:

```
Assignment
|-- Identifier(x)
'-- Add
    |-- Identifier(a)
    '-- Multiply
        |-- Identifier(b)
        '-- Identifier(c)
```

También puede representarse de manera compacta:

```

      =
    /  \
   x    +
      /  \
     a    *
      /  \
     b    c
```

El AST no necesita conservar explícitamente:

- el token de asignación como nodo independiente;
- el punto y coma;
- los paréntesis utilizados únicamente para agrupación;
- los no terminales intermedios de la gramática;
- cada producción empleada por el parser.

Sin embargo, deberá conservar suficiente información para reconstruir la estructura y realizar las siguientes etapas del compilador.

4.3. Precedencia representada en el AST

La estructura del AST deberá reflejar la precedencia de los operadores.

Para:

```
1 5 + 2 * 3
```

el AST esperado será equivalente a:

```
Add
|-- Integer(5)
'-- Multiply
    |-- Integer(2)
    '-- Integer(3)
```

La multiplicación deberá aparecer como descendiente de la suma, ya que tiene mayor precedencia. Para:

1 (5 + 2) * 3

el árbol deberá ser:

```
Multiply
|-- Add
|   |-- Integer(5)
|   '-- Integer(2)
'-- Integer(3)
```

Los paréntesis no necesariamente deberán representarse como nodos. Su efecto quedará reflejado en la estructura del árbol.

5. Descripción de la práctica

En esta práctica se extenderá el parser desarrollado anteriormente para construir el AST de los programas de MiniC.

La implementación deberá:

1. reutilizar el lexer completo desarrollado hasta la Práctica 2 y la interfaz incremental utilizada en la Práctica 3;
2. reutilizar el parser descendente recursivo de la Práctica 3;
3. diseñar una estructura de nodos para el AST;
4. representar programas como listas de sentencias;
5. construir nodos para declaraciones;
6. construir nodos para asignaciones;
7. construir nodos para sentencias de impresión;
8. construir nodos para condicionales;
9. construir nodos para ciclos;
10. construir nodos para bloques;
11. construir nodos para expresiones;
12. representar operadores unarios y binarios;
13. conservar información de línea y columna;
14. imprimir el AST;
15. liberar completamente la memoria utilizada;
16. continuar reportando errores léxicos y sintácticos.

El AST únicamente deberá construirse para las partes de la entrada que hayan sido reconocidas correctamente.

Un programa que contenga errores no deberá reportarse como válido.

6. Funcionalidades heredadas

La implementación deberá conservar todas las funcionalidades desarrolladas en las prácticas anteriores.

Esto incluye:

- lectura del archivo fuente;
- procesamiento de argumentos;
- sistema de tokens;
- seguimiento de línea y columna;
- reconocimiento completo de tokens de MiniC;
- manejo de comentarios;
- detección de errores léxicos;
- parser descendente recursivo;
- token de anticipación;
- reconocimiento de declaraciones;
- reconocimiento de asignaciones;
- reconocimiento de sentencias de impresión;
- reconocimiento de condicionales;
- reconocimiento de ciclos;
- reconocimiento de bloques;
- reconocimiento de expresiones;
- precedencia y asociatividad de operadores;
- detección de errores sintácticos;
- recuperación básica ante errores;
- consumo completo de la entrada;
- compilación mediante `Makefile`.

Las funcionalidades anteriores podrán evaluarse nuevamente.

No deberá implementarse un nuevo parser independiente.

La construcción del AST deberá integrarse al parser desarrollado en la Práctica 3.

7. Alcance del Árbol de Sintaxis Abstracta

El AST deberá representar las estructuras sintácticas incluidas en la gramática de MiniC utilizada en esta práctica.

Como mínimo, deberá ser posible representar:

- un programa;
- una lista de sentencias;
- declaraciones de variables;
- declaraciones con inicialización;
- asignaciones;
- sentencias de impresión;
- sentencias condicionales;
- ramas `else`;
- ciclos `while`;
- bloques;
- literales enteros;
- literales booleanos;
- identificadores;
- operadores aritméticos;
- operadores relacionales;
- operadores de igualdad;
- operadores lógicos;
- operador negativo unario.

En esta práctica no deberán representarse:

- tipos inferidos;
- referencias a símbolos;
- direcciones de memoria;
- valores calculados;
- instrucciones de código intermedio;
- información de generación de código;
- optimizaciones.

Estos elementos corresponden a etapas posteriores.

8. Tipos de nodos

La implementación deberá definir un conjunto de tipos de nodos que permita distinguir las construcciones del lenguaje.

Una enumeración posible es:

```
1 typedef enum {  
2     AST_PROGRAM,  
3     AST_BLOCK,  
4  
5     AST_VAR_DECL,  
6     AST_ASSIGNMENT,  
7     AST_PRINT,  
8     AST_IF,  
9     AST_WHILE,  
10  
11     AST_BINARY_EXPR,  
12     AST_UNARY_EXPR,  
13  
14     AST_IDENTIFIER,  
15     AST_INTEGER_LITERAL,  
16     AST_BOOLEAN_LITERAL  
17 } ASTNodeType;
```

Los nombres exactos podrán variar, siempre que:

- sean claros;
- sean consistentes;
- permitan identificar cada construcción;
- no mezclen categorías incompatibles;
- sean utilizados de manera uniforme.

También podrá utilizarse un tipo de nodo diferente para cada operador.

Por ejemplo:

```
1 AST_ADD
2 AST_SUBTRACT
3 AST_MULTIPLY
4 AST_DIVIDE
5 AST_LESS
6 AST_EQUAL
7 AST_AND
8 AST_OR
```

Ambos diseños serán válidos si están correctamente documentados y permiten imprimir y liberar el árbol.

9. Representación de operadores

Si se utiliza un nodo general para expresiones binarias, deberá existir una enumeración para identificar el operador.

Por ejemplo:

```
1 typedef enum {
2     OP_ADD,
3     OP_SUBTRACT,
4     OP_MULTIPLY,
5     OP_DIVIDE,
6
7     OP_LESS,
8     OP_LESS_EQUAL,
9     OP_GREATER,
10    OP_GREATER_EQUAL,
11
12    OP_EQUAL,
13    OP_NOT_EQUAL,
14
15    OP_AND,
16    OP_OR
17 } BinaryOperator;
```

Para operadores unarios podrá utilizarse:

```
1 typedef enum {
2     OP_NEGATE
3 } UnaryOperator;
```

El AST no deberá almacenar únicamente el lexema del operador sin una clasificación clara.

10. Estructura de los nodos

Se recomienda utilizar una estructura general con una unión que almacene la información específica de cada tipo de nodo.

Una posible implementación es:

```
1 typedef struct ASTNode ASTNode;
2
3 typedef struct {
4     ASTNode **items;
5     size_t count;
6     size_t capacity;
7 } ASTNodeList;
8
9 struct ASTNode {
10     ASTNodeType type;
11     int line;
12     int column;
13
14     union {
15         struct {
16             ASTNodeList statements;
17         } program;
18
19         struct {
20             ASTNodeList statements;
21         } block;
22
23         struct {
24             char *name;
25             TokenType declared_type;
26             ASTNode *initializer;
27         } variable_declaration;
28
29         struct {
30             char *name;
31             ASTNode *value;
32         } assignment;
33
34         struct {
35             ASTNode *expression;
36         } print_statement;
37
38         struct {
39             ASTNode *condition;
40             ASTNode *then_branch;
41             ASTNode *else_branch;
42         } if_statement;
43
44         struct {
45             ASTNode *condition;
46             ASTNode *body;
47         } while_statement;
```

```
48
49     struct {
50         BinaryOperator operator;
51         ASTNode *left;
52         ASTNode *right;
53     } binary;
54
55     struct {
56         UnaryOperator operator;
57         ASTNode *operand;
58     } unary;
59
60     struct {
61         char *name;
62     } identifier;
63
64     struct {
65         char *lexeme;
66     } integer_literal;
67
68     struct {
69         int value;
70     } boolean_literal;
71 } data;
72 };
```

La estructura anterior es únicamente una referencia.

El equipo podrá diseñar una representación diferente, siempre que:

- permita representar todas las construcciones requeridas;
- mantenga una separación clara entre los tipos de nodos;
- conserve la posición del código fuente;
- pueda imprimirse;
- pueda liberarse completamente;
- no dependa de variables globales innecesarias.

11. Información de posición

Cada nodo del AST deberá conservar la línea y la columna iniciales de la construcción que representa.

Se utilizará la misma convención establecida en las prácticas anteriores: las líneas comienzan en 1 y las columnas en 0.

La posición de cada nodo se determinará de la siguiente manera:

- Program: Posición de la primera sentencia; si el programa está vacío, posición de TOKEN_EOF;
- Block: posición de LBRACE;

- declaración de variable: posición de token IDENTIFIER declarado;
- asignación: posición del token IDENTIFIER ubicado como destino;
- sentencia print: posición del token PRINT;
- sentencia condicional: posición del token IF;
- ciclo: posición del token WHILE;
- expresión binaria: posición del token correspondiente al operador;
- expresión unaria: posición del token MINUS;
- identificador: posición de su token IDENTIFIER;
- literal entero o booleano: posición del token que lo representa;

Cuando una expresión se encuentre entre paréntesis, no será necesario crear un nodo adicional. El nodo de la expresión conservará su posición original.

Estas posiciones no deberán imprimirse como parte del formato canónico del AST, pero deberán permanecer almacenadas para que las etapas posteriores puedan producir diagnósticos precisos. Por ejemplo, la posición almacenada en el nodo de una declaración deberá corresponder al identificador declarado, ya que será utilizada para reportar redeclaraciones.

12. Listas de nodos

Un programa y un bloque pueden contener una cantidad variable de sentencias.

Por esta razón deberá implementarse una estructura dinámica para almacenar listas de nodos.

Una representación posible es:

```
1 typedef struct {  
2     ASTNode **items;  
3     size_t count;  
4     size_t capacity;  
5 } ASTNodeList;
```

La implementación deberá proporcionar funciones equivalentes a:

```
1 void ast_node_list_init(ASTNodeList *list);  
2  
3 int ast_node_list_append(  
4     ASTNodeList *list,  
5     ASTNode *node  
6 );  
7  
8 void ast_node_list_destroy(ASTNodeList *list);
```

La estructura deberá crecer dinámicamente cuando sea necesario.

No deberá establecerse un límite fijo pequeño para la cantidad de sentencias.

13. Construcción de nodos

Se recomienda proporcionar funciones constructoras para evitar inicializaciones incompletas o inconsistentes.

Ejemplos:

```
1  ASTNode *ast_create_program(  
2      ASTNodeList statements  
3  );  
4  
5  ASTNode *ast_create_variable_declaration(  
6      TokenType declared_type,  
7      const char *name,  
8      ASTNode *initializer,  
9      int line,  
10     int column  
11 );  
12  
13 ASTNode *ast_create_assignment(  
14     const char *name,  
15     ASTNode *value,  
16     int line,  
17     int column  
18 );  
19  
20 ASTNode *ast_create_binary(  
21     BinaryOperator operator,  
22     ASTNode *left,  
23     ASTNode *right,  
24     int line,  
25     int column  
26 );  
27  
28 ASTNode *ast_create_integer(  
29     const char *lexeme,  
30     int line,  
31     int column  
32 );  
33  
34 ASTNode *ast_create_identifier(  
35     const char *name,  
36     int line,  
37     int column  
38 );
```

Las funciones constructoras deberán:

1. reservar memoria;
2. validar el resultado de la reserva;
3. inicializar completamente el nodo;
4. copiar las cadenas que deban sobrevivir a los tokens;

5. establecer correctamente sus hijos;
6. conservar la información de posición.

14. Propiedad de la memoria

La implementación deberá definir un único propietario para cada recurso dinámico.

La propiedad de los tokens continuará regida por el contrato documentado en la Práctica 3. Antes de liberar o reemplazar un token, el parser deberá copiar al AST toda cadena que necesite sobrevivir.

Para el AST se utilizará la siguiente política:

- el AST deberá almacenar copias propias de los nombres de identificadores y de los lexemas que necesite conservar;
- el AST no deberá almacenar apuntadores hacia lexemas temporales pertenecientes al lexer o al parser;
- cada nodo es propietario de todos sus nodos hijos;
- los nodos Program y Block serán propietarios de sus listas de sentencias;
- el nodo raíz será propietario de todo el árbol;
- liberar el nodo raíz deberá liberar recursivamente todos los nodos, cadenas, listas y arreglos que formen parte del AST.

Cuando un constructor reciba nodos hijos, la transferencia de propiedad se realizará únicamente si el constructor termina correctamente:

- si el constructor devuelve un nodo válido, éste adquiere la propiedad de sus hijos;
- si el constructor devuelve NULL, los hijos continuarán perteneciendo al código que realizó la llamada y deberán ser liberados por éste.

La misma regla se aplicará a las listas:

- si `ast_node_list_append` termina correctamente, la lista adquiere la propiedad del nodo agregado;
- si la operación falla, la propiedad del nodo permanece en quien realizó la llamada;
- `ast_node_list_destroy` deberá liberar todos los nodos contenidos, el arreglo interno y dejar la lista en un estado vacío.

La función

```
1 void ast_destroy(ASTNode *node);
```

deberá aceptar NULL. En ese caso no realizará ninguna operación.

Cuando ocurra un error durante la construcción, deberán liberarse todos los nodos parciales que ya no formen parte de un árbol válido. No deberán producirse fugas, liberaciones dobles, punteros colgantes ni accesos posteriores a la liberación.

15. Modificación de las funciones del parser

Las funciones de reconocimiento deberán modificarse para devolver nodos.

Por ejemplo:

```
1 ASTNode *parse_program(Parser *parser);
2
3 ASTNode *parse_statement(Parser *parser);
4
5 ASTNode *parse_declaration(Parser *parser);
6
7 ASTNode *parse_assignment(Parser *parser);
8
9 ASTNode *parse_print_statement(Parser *parser);
10
11 ASTNode *parse_if_statement(Parser *parser);
12
13 ASTNode *parse_while_statement(Parser *parser);
14
15 ASTNode *parse_block(Parser *parser);
16
17 ASTNode *parse_expression(Parser *parser);
```

Las funciones de expresiones también deberán devolver nodos:

```
1 ASTNode *parse_or_expression(Parser *parser);
2
3 ASTNode *parse_and_expression(Parser *parser);
4
5 ASTNode *parse_equality_expression(Parser *parser);
6
7 ASTNode *parse_comparison_expression(
8     Parser *parser
9 );
10
11 ASTNode *parse_additive_expression(
12     Parser *parser
13 );
14
15 ASTNode *parse_multiplicative_expression(
16     Parser *parser
17 );
18
19 ASTNode *parse_unary_expression(Parser *parser);
20
21 ASTNode *parse_primary(Parser *parser);
```

Cuando una función reconozca correctamente su producción, deberá devolver el nodo correspondiente.

Si ocurre un error que impida construir la producción, deberá devolver:

```
1 NULL
```

o utilizar otra estrategia claramente documentada.

16. Construcción del nodo programa

La producción:

```
program ::= statement* TOKEN_EOF ;
```

deberá construir un nodo raíz de tipo programa.

El nodo deberá contener una lista ordenada de las sentencias reconocidas.

Ejemplo:

```
1 int x;  
2 x = 5;  
3 print(x);
```

podrá producir:

```
Program  
|-- VariableDeclaration  
|   |-- Type(int)  
|   '-- Identifier(x)  
|  
|-- Assignment  
|   |-- Identifier(x)  
|   '-- Integer(5)  
|  
'-- Print  
    '-- Identifier(x)
```

El orden de las sentencias deberá conservarse.

17. Construcción de declaraciones

La producción:

```
declaration ::= type IDENTIFIER  
               ( ASSIGN expression )?  
               SEMICOLON ;
```

deberá construir un nodo de declaración.

Para:

```
1 int contador;
```

el árbol podrá ser:

```
VariableDeclaration  
|-- Type(int)  
'-- Identifier(contador)
```

Para:

```
1 int contador = 10;
```

deberá incluirse la expresión inicializadora:

```
VariableDeclaration
|-- Type(int)
|-- Identifier(contador)
'-- Initializer
    '-- Integer(10)
```

La ausencia de inicializador deberá representarse mediante un apuntador nulo o un mecanismo equivalente.

No deberá crearse un nodo de asignación independiente para la inicialización, salvo que el diseño esté claramente justificado.

18. Construcción de asignaciones

La producción:

```
assignment ::= IDENTIFIER ASSIGN
              expression SEMICOLON ;
```

deberá producir un nodo de asignación.

Para:

```
1 x = 5 + 3;
```

el árbol deberá ser equivalente a:

```
Assignment
|-- Identifier(x)
'-- Add
    |-- Integer(5)
    '-- Integer(3)
```

El identificador ubicado en el lado izquierdo deberá conservarse como destino de la asignación.

19. Construcción de sentencias de impresión

La producción:

```
print_statement ::= PRINT LPAREN
                  expression RPAREN
                  SEMICOLON ;
```

deberá producir un nodo de impresión.

Para:

```
1 print(x + 1);
```

el árbol será equivalente a:

```
Print
|-- Add
|   |-- Identifier(x)
|   |-- Integer(1)
```

Los paréntesis y el punto y coma no deberán almacenarse como nodos.

20. Construcción de condicionales

La producción:

```
if_statement ::= IF LPAREN expression RPAREN
               statement
               ( ELSE statement )? ;
```

deberá producir un nodo con:

- condición;
- rama verdadera;
- rama falsa opcional.

Para:

```
1 if (x > 0) {
2     print(x);
3 }
```

el árbol podrá ser:

```
If
|-- Condition
|   |-- Greater
|       |-- Identifier(x)
|       |-- Integer(0)
|-- Then
|   |-- Block
|       |-- Print
|           |-- Identifier(x)
```

Para:

```

1 if (activo) {
2     print(1);
3 } else {
4     print(0);
5 }

```

deberá representarse también la rama falsa:

```

If
|-- Condition
|   '-- Identifier(activo)
|
|-- Then
|   '-- Block
|       '-- Print
|           '-- Integer(1)
|
|-- Else
|   '-- Block
|       '-- Print
|           '-- Integer(0)

```

Cuando no exista una rama `else`, el campo correspondiente podrá contener `NULL`.

21. Construcción de ciclos

La producción:

```

while_statement ::= WHILE LPAREN expression
                  RPAREN statement ;

```

deberá producir un nodo con: condición y cuerpo. Para:

```

1 while (x > 0) {
2     x = x - 1;
3 }

```

el árbol esperado será equivalente a:

```

While
|-- Condition
|   '-- Greater
|       |-- Identifier(x)
|       '-- Integer(0)
|
|-- Body
|   '-- Block
|       '-- Assignment
|           |-- Identifier(x)
|           '-- Subtract
|               |-- Identifier(x)
|               '-- Integer(1)

```

22. Construcción de bloques

La producción:

```
block ::= LBRACE statement* RBRACE ;
```

deberá producir un nodo que contenga una lista de sentencias.

Para:

```
1 {  
2     int x;  
3     x = 10;  
4 }
```

el árbol será equivalente a:

```
Block  
|-- VariableDeclaration  
|   |-- Type(int)  
|   '-- Identifier(x)  
|  
'-- Assignment  
    |-- Identifier(x)  
    '-- Integer(10)
```

Un bloque vacío deberá representarse como un nodo con una lista vacía:

```
Block
```

23. Construcción de expresiones primarias

La producción:

```
primary ::= INTEGER  
          | TRUE  
          | FALSE  
          | IDENTIFIER  
          | LPAREN expression RPAREN ;
```

deberá construir:

- un nodo de literal entero;
- un nodo de literal booleano;
- un nodo de identificador;
- el nodo de la expresión contenida entre paréntesis.

Ejemplos:

```
1 10
2 true
3 false
4 contador
```

deberán representarse como:

```
Integer(10)

Boolean(true)

Boolean(false)

Identifier(contador)
```

Para:

```
1 (5 + 3)
```

no será obligatorio crear un nodo específico de paréntesis.

Deberá devolverse el nodo correspondiente a:

```
Add
|-- Integer(5)
'-- Integer(3)
```

24. Construcción de expresiones unarias

La producción:

```
unary_expression ::= MINUS unary_expression
                  | primary ;
```

deberá construir un nodo unario para el operador negativo.

Para:

```
1 -10
```

el árbol será:

```
Negate
'-- Integer(10)
```

Para:

```
1 --x
```

la gramática permite una estructura equivalente a:

```
Negate
'-- Negate
    '-- Identifier(x)
```

La validez semántica o utilidad de esta expresión no deberá evaluarse todavía.

25. Construcción de expresiones binarias

Cada operador binario deberá producir un nodo con dos hijos:

- operando izquierdo;
- operando derecho.

Por ejemplo:

```
1 a + b
```

deberá producir:

```
Add
|-- Identifier(a)
'-- Identifier(b)
```

Para construir correctamente la asociatividad izquierda, una función podrá seguir una estrategia equivalente a:

```
1 ASTNode *parse_additive_expression(
2     Parser *parser
3 ) {
4     ASTNode *left =
5         parse_multiplicative_expression(parser);
6
7     while (
8         check(parser, PLUS) ||
9         check(parser, MINUS)
10    ) {
11        Token operator_token = parser->current;
12        advance(parser);
13
14        ASTNode *right =
15            parse_multiplicative_expression(
16                parser
17            );
18
19        left = ast_create_binary(
20            token_to_binary_operator(
21                operator_token.type
22            ),
23            left,
24            right,
25            operator_token.line,
26            operator_token.column
27        );
28    }
29
30    return left;
31 }
```

El código anterior es ilustrativo.

La implementación deberá validar los errores de memoria y los nodos nulos.

26. Asociatividad

La estructura del AST deberá reflejar correctamente la asociatividad.

Para:

```
1 10 - 5 - 2
```

la asociatividad izquierda implica:

```
Subtract
|-- Subtract
|   |-- Integer(10)
|   '-- Integer(5)
|
'-- Integer(2)
```

No deberá construirse:

```
Subtract
|-- Integer(10)
'-- Subtract
    |-- Integer(5)
    '-- Integer(2)
```

Para el operador unario negativo deberá conservarse la asociatividad derecha establecida por la gramática.

27. Manejo de errores durante la construcción

La construcción del AST no reemplaza el manejo de errores de la Práctica 3.

Si se detecta un error léxico o sintáctico:

- deberá reportarse con línea y columna;
- el programa no deberá declararse correcto;
- deberá evitarse construir nodos con información incompleta;
- deberá liberarse cualquier nodo parcial que ya no pueda utilizarse;
- deberá aplicarse la estrategia de recuperación;
- deberá evitarse el acceso a apuntadores nulos.

Por ejemplo, para:

```
1 x = 5 + ;
```

no deberá construirse un nodo binario válido con un operando derecho inexistente.

La función podrá devolver NULL y permitir que el nivel superior libere los nodos parciales.

La estrategia exacta deberá documentarse.

28. Construcción parcial y recuperación

La construcción del AST no reemplaza la recuperación ante errores implementada en la Práctica 3.

Cuando se detecte un error léxico o sintáctico, el parser deberá intentar recuperarse y continuar el análisis para localizar errores posteriores. Durante este proceso podrá construir temporalmente nodos correspondientes a producciones reconocidas correctamente.

Sin embargo, si se detecta al menos un error:

- el programa completo deberá marcarse como inválido;
- no deberán incorporarse al AST nodos corruptos o incompletos;
- cualquier nodo parcial que ya no pueda utilizarse deberá liberarse;
- el AST parcial no deberá entregarse a etapas posteriores;
- el AST no deberá imprimirse;
- no deberá imprimirse el mensaje de programa correcto;
- los diagnósticos deberán enviarse exclusivamente a stderr;
- stdout deberá permanecer vacío;
- el proceso deberá finalizar con un código distinto de cero.

Una función del parser podrá devolver NULL cuando no sea posible construir correctamente la producción correspondiente. La función que realizó la llamada deberá detectar este resultado y liberar los nodos parciales cuya propiedad todavía conserve.

Al finalizar el análisis de un programa inválido, cualquier árbol parcial que permanezca en memoria deberá destruirse antes de terminar el proceso.

29. Impresión del AST

La implementación deberá incluir una función que imprima el AST de manera jerárquica y determinista. Una interfaz posible es:

```
1 void ast_print(const ASTNode *node);
```

Podrá utilizarse una función auxiliar que reciba el nivel de profundidad:

```
1 void ast_print_indented(  
2     const ASTNode *node,  
3     int depth  
4 );
```

La función deberá imprimir:

- el tipo de cada nodo;
- los nombres y valores relevantes;
- la relación jerárquica entre padres e hijos;

- los operandos de izquierda a derecha;
- las sentencias en el orden en que aparecen en el programa.

La impresión deberá utilizar exclusivamente el formato canónico definido en la sección 30.

Los diagramas con conectores utilizados en las secciones anteriores son únicamente representaciones conceptuales. No deberán utilizarse como formato de salida del ejecutable.

La función `ast_print` imprimirá únicamente el nodo recibido y sus descendientes. El encabezado:

```
1 AST:
```

y el mensaje final:

```
1 Programa sintacticamente correcto.
```

serán responsabilidad del programa principal.

El AST sólo deberá imprimirse cuando el archivo completo no contenga errores léxicos ni sintácticos.

30. Formato canónico de impresión

Para facilitar las pruebas automáticas, la impresión del AST deberá utilizar exactamente dos espacios por nivel de profundidad y un elemento por línea.

No deberán utilizarse tabuladores, conectores gráficos, líneas vacías, direcciones de memoria, tokens de puntuación ni mensajes de depuración.

La salida de un programa válido deberá seguir esta estructura:

```
AST:
Program
  <sentencias>
Programa sintacticamente correcto.
```

Las etiquetas canónicas serán las siguientes:

- | | | |
|--|-------------|------------------------|
| ■ Program | ■ Else | ■ GreaterEqual |
| ■ Block | ■ While | ■ Equal |
| ■ VariableDeclaration(int,
<nombre>) | ■ Body | ■ NotEqual |
| ■ VariableDeclaration(bool,
<nombre>) | ■ Add | ■ And |
| ■ Assignment(<nombre>) | ■ Subtract | ■ Or |
| ■ Print | ■ Multiply | ■ Negate |
| ■ If | ■ Divide | ■ Identifier(<nombre>) |
| ■ Condition | ■ Less | ■ Integer(<lexema>) |
| ■ Then | ■ LessEqual | ■ Boolean(true) |
| | ■ Greater | ■ Boolean(false) |

Las etiquetas Condition, Then, Else y Body se utilizarán únicamente para identificar la función de los hijos durante la impresión. No será obligatorio representarlas como nodos reales dentro del AST.

Se aplicarán las siguientes reglas:

- Program se imprimirá en el nivel 0.
- Las sentencias del programa se imprimirán como hijos de Program.
- Un programa vacío imprimirá únicamente Program.
- Block imprimirá sus sentencias en orden.
- Un bloque vacío imprimirá únicamente Block.
- Una declaración sin inicialización no tendrá hijos.
- Una declaración con inicialización imprimirá la expresión inicializadora como hija.
- Assignment(<nombre>) imprimirá la expresión asignada como hija.
- Print imprimirá su expresión como hija.
- If imprimirá, en este orden, Condition, Then y, cuando exista, Else.
- While imprimirá, en este orden, Condition y Body.
- Una expresión binaria imprimirá primero el operando izquierdo y después el derecho.
- Negate imprimirá su operando como único hijo.
- Los identificadores, enteros y booleanos no tendrán hijos.
- La salida deberá terminar con un salto de línea.

Ejemplo:

```
AST:
Program
  VariableDeclaration(int, x)
    Integer(10)
  Assignment(x)
    Add
      Identifier(x)
      Integer(1)
  Print
    Identifier(x)
Programa sintacticamente correcto.
```

31. Liberación del AST

Deberá implementarse una función que libere recursivamente todo el árbol.

Una interfaz posible es:

```
1 void ast_destroy(ASTNode *node);
```

La función deberá liberar, según el tipo del nodo:

- cadenas copiadas;
- nodos hijos;
- listas dinámicas;
- arreglos internos;
- el propio nodo.

Por ejemplo, para un nodo binario:

```
1 ast_destroy(node->data.binary.left);  
2 ast_destroy(node->data.binary.right);  
3 free(node);
```

Para un nodo programa o bloque deberá recorrerse toda la lista de sentencias.

Liberar el nodo raíz deberá liberar el árbol completo.

No deberán existir:

- fugas de memoria;
- liberaciones dobles;
- accesos después de liberar;
- cadenas sin propietario;
- nodos huérfanos.

32. Resultado de una ejecución válida

Para el archivo:

```
1 int x = 5;  
2 x = x + 3;  
3 print(x);
```

la ejecución deberá producir exactamente:

```
AST:  
Program  
  VariableDeclaration(int, x)  
    Integer(5)  
  Assignment(x)  
    Add  
      Identifier(x)  
      Integer(3)  
  Print  
    Identifier(x)  
Programa sintacticamente correcto.
```

El programa deberá finalizar con código:

```
0
```

33. Resultado de una ejecución inválida

Para:

```
1 int x = ;  
2 print(x);
```

stderr deberá contener un diagnóstico equivalente a:

```
Error sintactico [1:8]:  
se esperaba una expresion,  
pero se encontro ';'.
```

No deberá imprimirse:

```
Programa sintacticamente correcto.
```

No deberá imprimirse el AST parcial.

El programa deberá finalizar con un código distinto de cero.

34. Ejemplo completo

Considere el siguiente programa:

```
1 int contador = 3;
2 bool activo = true;
3
4 while (contador > 0 && activo) {
5     print(contador);
6     contador = contador - 1;
7 }
8
9 activo = false;
```

El ejecutable deberá producir exactamente:

AST:

Program

VariableDeclaration(int, contador)

Integer(3)

VariableDeclaration(bool, activo)

Boolean(true)

While

Condition

And

Greater

Identifier(contador)

Integer(0)

Identifier(activo)

Body

Block

Print

Identifier(contador)

Assignment(contador)

Subtract

Identifier(contador)

Integer(1)

Assignment(activo)

Boolean(false)

Programa sintacticamente correcto.

La estructura refleja que:

- las declaraciones pertenecen directamente al programa;
- la condición del ciclo es una conjunción;
- la comparación `contador > 0` es el operando izquierdo de And;
- el cuerpo del ciclo es un bloque;
- las sentencias del bloque conservan su orden;
- la resta es hija de la asignación;
- la última asignación pertenece al programa y no al bloque del ciclo.

No deberán aparecer nodos para paréntesis, puntos y coma ni demás tokens utilizados únicamente para describir la sintaxis concreta.

35. Estructura del proyecto

El proyecto deberá extender la organización utilizada en la Práctica 3.

Como mínimo, deberá utilizarse una estructura equivalente a:

```
.
|-- include/
|   |-- lexer/
|   |   |-- lexer.h
|   |   '-- token.h
|   |
|   |-- parser/
|   |   '-- parser.h
|   |
|   '-- ast/
|       '-- ast.h
|
|-- src/
|   |-- lexer/
|   |   |-- lexer.c
|   |   '-- token.c
|   |
|   |-- parser/
|   |   '-- parser.c
|   |
|   |-- ast/
|   |   '-- ast.c
|   |
|   '-- main.c
|
|-- tests/
|
|-- Makefile
|-- README.md
'-- CHANGELOG.md
```

Podrán agregarse archivos adicionales, por ejemplo:

```
include/ast/ast_printer.h
src/ast/ast_printer.c
```

si se desea separar la construcción, impresión y liberación del árbol.

No deberá colocarse toda la implementación del AST dentro de `parser.c` ni dentro de `main.c`.

36. Responsabilidades de los módulos

La separación mínima esperada será:

36.1. Lexer

Responsable de:

- leer caracteres;
- conservar línea y columna;
- reconocer lexemas;
- reconocer y producir tokens ERROR.
- producir tokens;

36.2. Parser

Responsable de:

- consumir tokens;
- reportar errores sintácticos;
- emitir un único diagnóstico léxico por cada token ERROR consumido, sin reportar además ese mismo token como un error sintáctico;
- aplicar recuperación;
- solicitar la creación de nodos;
- reconocer producciones;
- devolver el nodo raíz.

36.3. AST

Responsable de:

- definir los tipos de nodos;
- imprimir el árbol;
- construir nodos;
- liberar memoria.
- administrar listas;

36.4. Programa principal

El archivo `main.c` deberá limitarse principalmente a:

1. validar argumentos;
2. inicializar el lexer;
3. inicializar el parser;
4. ejecutar el análisis;
5. obtener el nodo raíz;
6. imprimir el resultado;
7. liberar el árbol;
8. liberar los demás recursos;
9. devolver el código de salida.

37. Compilación y ejecución

El proyecto deberá compilarse mediante:

```
make
```

El ejecutable generado deberá llamarse:

```
minic
```

La ejecución deberá realizarse mediante:

```
./minic programa.mc
```

El comando:

```
make clean
```

deberá eliminar los archivos generados durante la compilación.

Se recomienda utilizar como mínimo:

```
-std=c11 -Wall -Wextra -Wpedantic
```

El proyecto deberá compilar sin errores.

Los mensajes de advertencia podrán considerarse durante la evaluación de la calidad del código.

38. Pruebas

El proyecto deberá incluir pruebas que permitan verificar tanto la validación sintáctica como la estructura del AST.

Como mínimo, deberán probarse los siguientes casos:

- | | |
|--------------------------------------|---------------------------------|
| ■ archivo vacío; | ■ condicional con else ; |
| ■ programa con una sentencia; | ■ condicionales anidados; |
| ■ programa con múltiples sentencias; | ■ ciclo while ; |
| ■ declaración sin inicialización; | ■ ciclos anidados; |
| ■ declaración con inicialización; | ■ identificadores; |
| ■ declaración de tipo entero; | ■ literales enteros; |
| ■ declaración de tipo booleano; | ■ literales booleanos; |
| ■ asignación; | ■ negación unaria; |
| ■ impresión; | ■ suma y resta; |
| ■ bloque vacío; | ■ multiplicación y división; |
| ■ bloque con múltiples sentencias; | ■ operadores relacionales; |
| ■ bloques anidados; | ■ operadores de igualdad; |
| ■ condicional sin else ; | ■ operadores lógicos; |
| | ■ precedencia entre operadores; |

- asociatividad izquierda;
- paréntesis;
- expresiones profundamente anidadas;
- errores léxicos;
- errores sintácticos;
- recuperación ante errores;
- ausencia de nodos incompletos;
- impresión determinista;
- liberación del árbol;
- programas extensos.

Las pruebas deberán verificar no solamente que un programa sea aceptado, sino que el árbol resultante tenga la estructura correcta.

Por ejemplo, para:

```
1 x = 2 + 3 * 4;
```

no será suficiente aceptar la entrada.

La prueba deberá verificar que la multiplicación sea descendiente de la suma.

39. Pruebas de memoria

La implementación deberá probarse con una herramienta de detección de errores de memoria, cuando se encuentre disponible en el entorno del curso.

Por ejemplo:

```
valgrind --leak-check=full \  
        --show-leak-kinds=all \  
        ./minic programa.mc
```

Se deberá verificar:

- ausencia de fugas de memoria;
- ausencia de accesos inválidos;
- ausencia de liberaciones dobles;
- liberación correcta ante programas válidos;
- liberación correcta ante programas inválidos;
- liberación correcta después de errores de construcción.

40. Restricciones

Para esta práctica:

- el lenguaje de implementación deberá ser C;
- deberá reutilizarse el lexer completo desarrollado hasta la Práctica 2 y la interfaz incremental utilizada en la Práctica 3;
- deberá reutilizarse el parser de la Práctica 3;
- el parser deberá continuar siendo descendente recursivo;

- no deberán utilizarse Lex, Flex, Yacc o Bison;
- deberá respetarse la gramática oficial de MiniC;
- deberá construirse un AST;
- no deberá construirse un CST completo;
- cada función principal del parser deberá devolver el nodo correspondiente;
- deberá conservarse la precedencia y asociatividad;
- deberá almacenarse información de línea y columna;
- deberá implementarse una lista dinámica de sentencias;
- deberá imprimirse el árbol;
- deberá liberarse completamente el AST;
- no deberán utilizarse tamaños fijos pequeños para limitar sentencias;
- no deberán utilizarse variables globales innecesarias;
- no deberán realizarse validaciones semánticas;
- no deberá implementarse una tabla de símbolos;
- no deberán verificarse tipos;
- no deberá generarse código;
- no deberá implementarse todo dentro de `main.c`;
- el módulo del AST deberá permanecer separado del parser;
- deberá utilizarse un `Makefile`;
- deberán incluirse pruebas;
- deberán respetarse los lineamientos generales de entrega.

41. Entregables

La entrega deberá incluir:

```
Practica04_EquipoXX/  
|-- README.md  
|-- CHANGELOG.md  
|-- reporte.pdf  
|-- Makefile  
|-- include/  
|   |-- lexer/  
|   |-- parser/  
|   '-- ast/  
|-- src/  
|   |-- lexer/
```

```
|  |-- parser/
|  '-- ast/
'-- tests/
```

Todos los elementos deberán entregarse de acuerdo con los lineamientos generales establecidos para el curso.

42. Criterios de evaluación

La evaluación se realizará de acuerdo con los criterios generales establecidos para la entrega de prácticas, considerando los objetivos y requerimientos específicos de la práctica.

La siguiente rúbrica describe los elementos considerados para la evaluación de la **Práctica 4**.

Aspecto	Criterios específicos	Ponderación
Correctitud de la implementación	Construcción correcta del nodo programa, declaraciones, asignaciones, impresión, condicionales, ciclos, bloques y expresiones; representación correcta de precedencia y asociatividad; impresión completa y determinista del árbol; conservación de línea y columna; manejo correcto de errores.	40 %
Diseño y calidad del código	Separación entre lexer, parser y AST; diseño claro de nodos; uso adecuado de uniones, enumeraciones y listas dinámicas; funciones constructoras; manejo de propiedad de memoria; liberación recursiva; ausencia de duplicación innecesaria, fugas y accesos inválidos.	20 %
Documentación	Cumplimiento de los lineamientos del reporte, <code>README.md</code> y <code>CHANGELOG.md</code> ; explicación del diseño del AST, tipos de nodos, propiedad de memoria, construcción desde el parser, formato de impresión y pruebas realizadas.	20 %
Compilación y ejecución	Compilación mediante el <code>Makefile</code> ; generación del ejecutable <code>minic</code> ; ejecución mediante la interfaz especificada; ausencia de errores de compilación; funcionamiento de <code>make clean</code> ; códigos de salida correctos y cumplimiento del formato de impresión.	20 %
Total		100 %

La presencia de funcionalidades no solicitadas no compensará errores en la construcción del AST. En particular:

- una impresión visualmente atractiva no compensará un árbol estructuralmente incorrecto;
- un parser que acepte programas válidos no será suficiente si no construye los nodos requeridos;
- la construcción de nodos no compensará fugas graves de memoria;
- la incorporación anticipada de análisis semántico no sustituirá los requerimientos de esta práctica.

43. Resultado esperado

Al finalizar esta práctica se deberá contar con un compilador de MiniC capaz de realizar el siguiente proceso:

```
Programa fuente
|
v
Analizador lexico
|
v
Secuencia de tokens
|
v
Analizador sintactico
|
|-- reconoce producciones
|-- valida la estructura
|-- reporta errores
'-- construye nodos
|
v
Arbol de Sintaxis Abstracta
|
|-- programa
|-- declaraciones
|-- asignaciones
|-- impresion
|-- condicionales
|-- ciclos
|-- bloques
|-- expresiones
'-- literales e identificadores
|
v
Impresion jerarquica
```

La Práctica 3 respondía:

¿El programa cumple con la gramática?

La Práctica 4 deberá responder además:

¿Cuál es la estructura abstracta del programa?

El AST producido servirá como entrada para las siguientes etapas del compilador:

```
AST
|
v
Análisis semántico
|
v
Tabla de símbolos y tipos
|
v
Generación de código
```

Por esta razón, la representación deberá ser:

- correcta;
- estable;
- modular;
- extensible;
- independiente de la impresión;
- segura respecto al manejo de memoria.

“Todo lo que tiene un principio tiene un final.”

— El Oráculo, *The Matrix Revolutions*